LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 12. Sets

Lecture: 128.Sets in Mathematics

Section 1: Lecture Summary

This lecture introduces the mathematical concept of sets, which directly relates to Python's set data type. It explains what a set is in mathematics: a collection of distinct elements without duplicates. The lecture covers key terminology such as subsets, supersets, proper subsets, and disjoint sets, using various example sets to illustrate these ideas. It also clarifies the relationships between sets, such as when one set is contained within another, and distinguishes between equal sets and proper subsets. The lecture concludes by noting that Python's set class implements these mathematical concepts and that future lessons will explore the corresponding Python methods.

Section 2: Key Concepts and Explanations

- **Set (Mathematical Concept)**

  - A set is a collection of unique elements (no duplicates).

  - Elements can be of any type but usually are homogeneous for clarity (e.g., all integers).

  - Example set: $S = \{1, 2, 3, \ldots, 10\}$.

- **Subset and Superset**

  - Subset ($\subseteq$): Set A is a subset of S if all elements of A are in S.

  - Superset ($\supseteq$): Set S is a superset of A if it contains all elements of A.

  - Example: $A = \{1, 2, 3, 5, 7\}$ is a subset of S, so S is a superset of A.

- **Proper Subset and Proper Superset**

  - A proper subset is a subset that is strictly contained in another set (i.e., it does not contain all elements of the superset).

  - \(A \) is a proper subset of \(S \) if \(A \subset S \) and \(A \neq S \).

  - If \(C = S \), then \(C \) is a subset and equal to \(S \), so not a proper subset.

- **Equal Sets**

  - Two sets are equal if they have exactly the same elements.

  - Equal sets are subsets of each other and supersets of each other.

- **Disjoint Sets**

  - Two sets are disjoint if they have no common elements.

  - Example: \(D = \{1, 2, 3, 4, 5\} \) and \(E = \{6, 7, 8, 9, 10\} \) are disjoint subsets of \(S \).

- **Terminology Symbols**

  - \(\subseteq\): subset or equal

  - \(\subset\): proper subset

  - \(\supseteq\): superset or equal

  - \(\supset\): proper superset

  - Disjoint sets have empty intersection.

- **Relation to Python**

  - Python’s set class models these mathematical set concepts.

  - Python sets do not allow duplicate elements.

- Python provides methods to work with subsets, supersets, intersections, and disjointness (to be covered in later lectures).

Section 3: Example Code and Use Cases

This lecture focused on theory and did not provide explicit Python code. However, based on the concepts explained, here is a relevant example demonstrating some of these concepts in Python:

Define sets

Check subsets

Proper subsets (in Python, no direct method, check with inequality)

Check supersets

Check disjoint sets

```
S = set(range(1, 11))  # S = {1,2,...,10}
A = {1, 2, 3, 5, 7}
B = {5, 7, 9, 10}
C = set(range(1, 11))  # Equal to S
D = {1, 2, 3, 4, 5}
E = {6, 7, 8, 9, 10}

print(A.issubset(S))  # True
print(B.issubset(S))  # True
print(C == S)         # True

print(A < S)          # True, proper subset
print(C < S)          # False, equal sets

print(S.issuperset(A))  # True

print(D.isdisjoint(E))  # True, no common elements
print(A.isdisjoint(B))  # False, common elements 5 and 7
```

This code illustrates how to represent sets, test subset and superset relations, equality, proper subsets, and disjointness using Python sets, reinforcing the lecture's core concepts.

Section 4: Key Takeaways

- A set is a collection of unique elements with no duplicates.

- Subset means all elements of one set exist in another; superset is the inverse relation.

- Proper subset excludes equality; equal sets are subsets of each other.

- Disjoint sets have no elements in common.

- The mathematical set notions directly inform Python’s set data structure.

- Python sets support subset, superset, equality, and disjointness checks via built-in methods.

- Understanding these mathematical foundations is essential for effective use of Python sets.